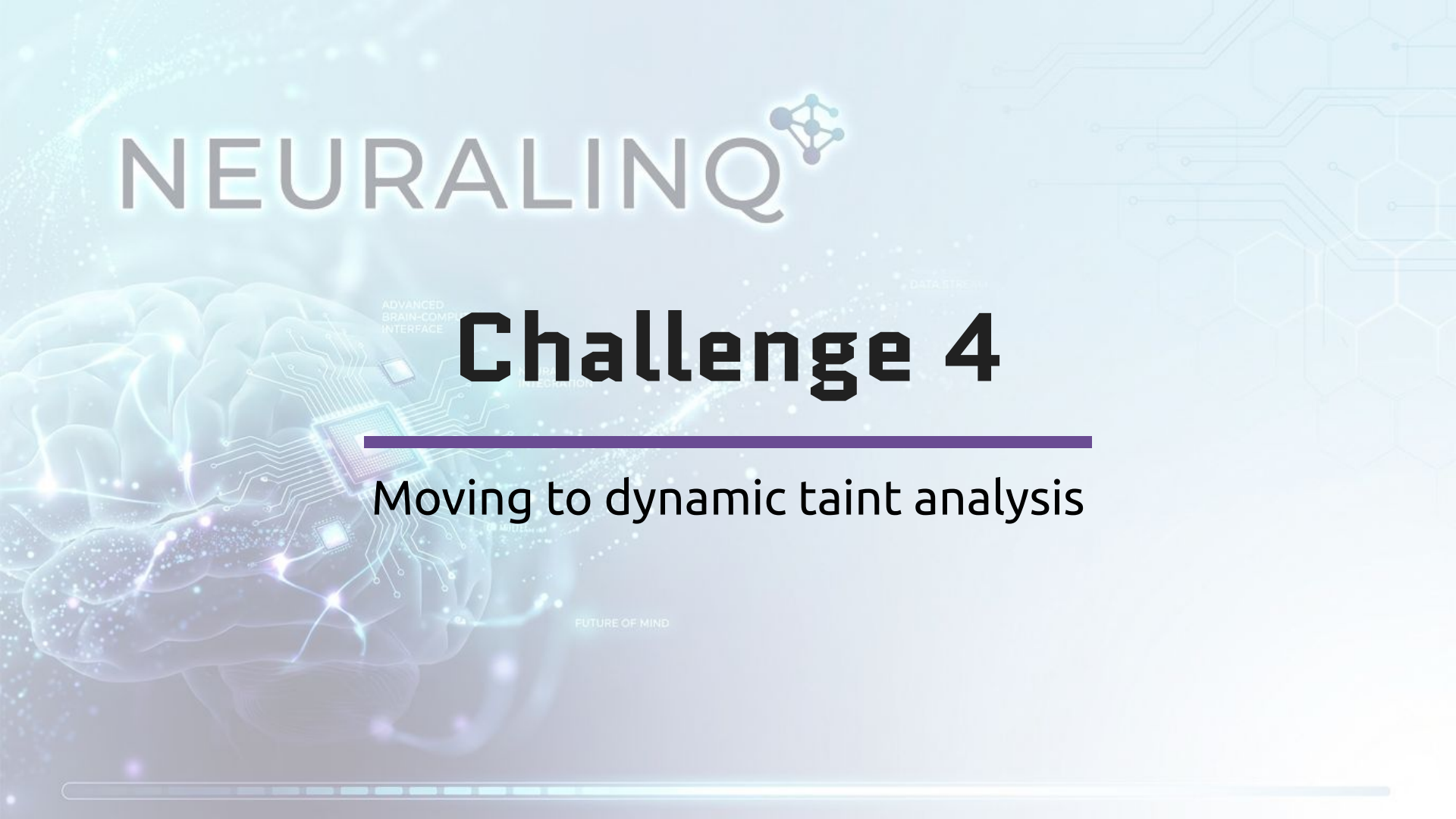




NEURALINQ



ADVANCED
BRAIN-COMPUTER
INTERFACE

Challenge 4

DATA STREAM

INTEGRATION

FUTURE OF MIND

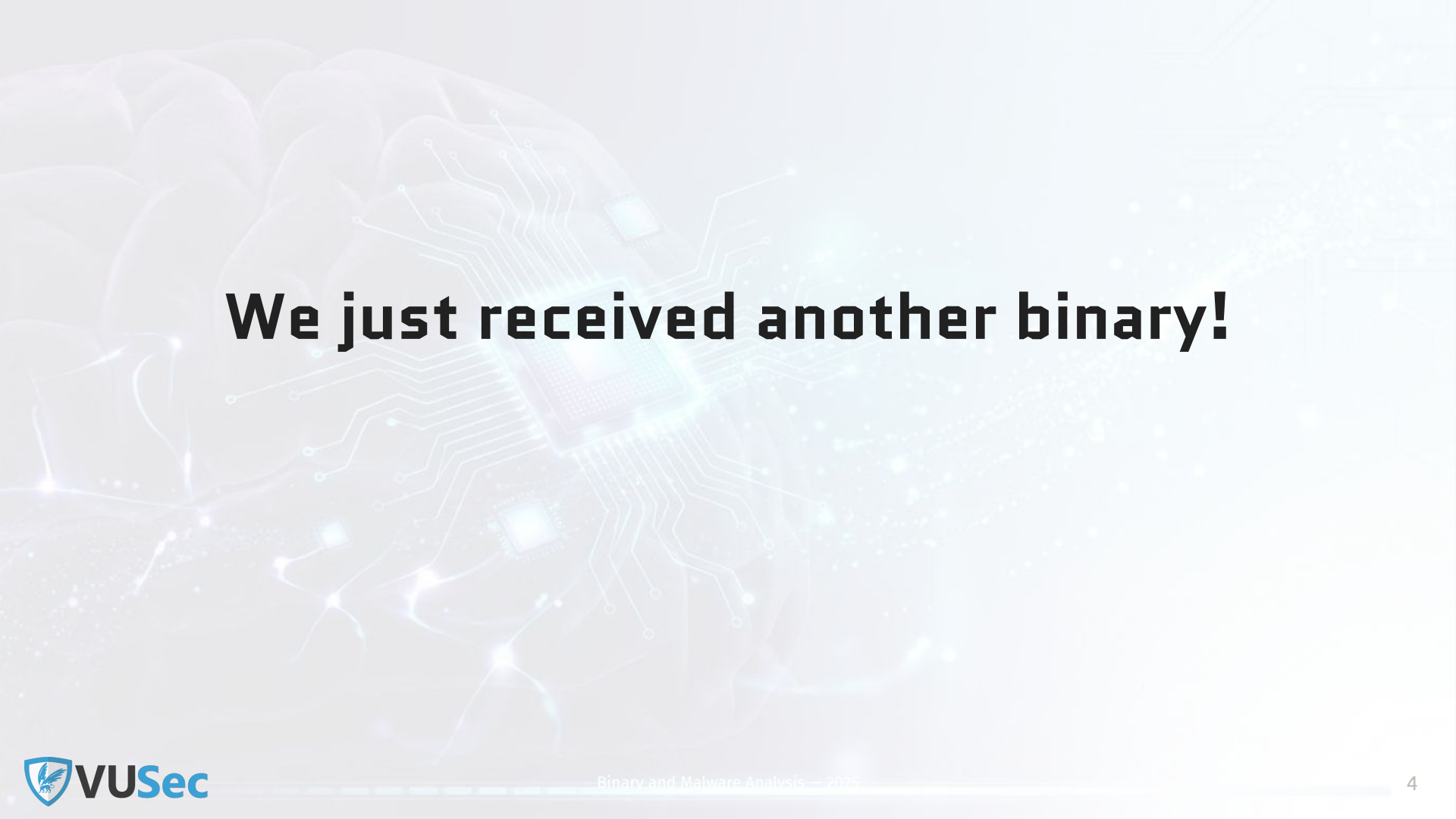
Moving to dynamic taint analysis

Thus far

- We investigated the Launcher of Neuraling to connect with the implants
 - Disabled anti-* checks...
 - Unpacked and repaired an inner binary...
 - Used the NLQLauncher to launch something?!

Thus far

- We investigated the Launcher of Neuraling to connect with the implants
 - Disabled anti-* checks...
 - Unpacked and repaired an inner binary...
 - Used the NLQLauncher to launch something?!
 - But there's more?



We just received another binary!

We just received another binary!

- Sender unclear...

We just received another binary!

- Sender unclear...
- Is it even a human?

We just received another binary!

- Sender unclear...
- Is it even a human?
- Seems that 'it', inside this box, wants to tell us something

We just received another binary!

- Sender unclear...
- Is it even a human?
- Seems that 'it', inside this box, wants to tell us something
- \$./NLQBox --help

NLQBox

\$./NLQBox -i <identity_file>

- It appears that the binary expects a “identity file”
- Without the correct identity binary is inaccessible...

NLQBox

\$./NLQBox -i <identity_file>

- It appears that the binary expects a “identity file”
- Without the correct identity binary is inaccessible...
- We are stuck here...
- Luckily we have you to help us!

NEURALINQ

Access your dump

Your task

ADVANCED BRAIN EXPLOIT VECTOR
BRAIN-COMPUTER
INTERFACE

NEURALINQ
INTEGRATION

ROOT ACCESS GRANTED
COMMAND_OVERRIDE_DET
NEURAL SUBJUGATION
NEURAL OPERCTIOO
NEURAL COVERRITEG_DET

CMD: HACK_BRAIN_FUNC
SYSTEM COMPROMISED
HACKER_ID: [REDACTED]
ACCESS: ALL
BRAIN_FUNCTION_OVERRIDE: ACTIVE

Why are we without

END_OF_FREE_WILL

BRAIN_EXPLOIT_VECTED

0101010001000101
1001101011001
0110001001010100
01001010011101
0100000010101
010101010001

CONTROL_UPLOAD: 91%

STATUS: IN CONTACT
ACCESS: ALL
BRAIN_FUNCTION_OVERRIDE

STATUS: BRAIN FLA
SYSTEM COMPROMISED
BRAIN_FUNCTION_OVERRIDE

CMD: HACK_BRAIN_FUNC
SYSTEM COMPROMISED
HACKER_ID: [REDACTED]
ACCESS: ALL
BRAIN_FUNCTION_OVERRIDE

CMD: HACK_BRAIN_FUNC
SYSTEM COMPROMISED
HACKER_ID: [REDACTED]
ACCESS: ALL
BRAIN_FUNCTION_OVERRIDE

ACCESS GRANTED

CMD: HACK_BRAIN_FUNC
SYSTEM COMPROMISED
HACKER_ID: [REDACTED]
ACCESS: ALL
BRAIN_FUNCTION_OVERRIDE: ACTIVE

Your task

- Write a **dynamic taint analysis engine**, and use it to obtain the secret identity-file.

Your task

- Write a **dynamic taint analysis engine**, and use it to obtain the secret identity-file.
- Once you recovered the identity-file, the thing in the box will trust you to tell its inner secrets.

Your task

- Write a **dynamic taint analysis engine**, and use it to obtain the secret identity-file.
- Once you recovered the identity-file, the thing in the box will trust you to tell its inner secrets.
- You must write your own PIN tool, based on the framework on Canvas (as discussed during the lecture).

Your task

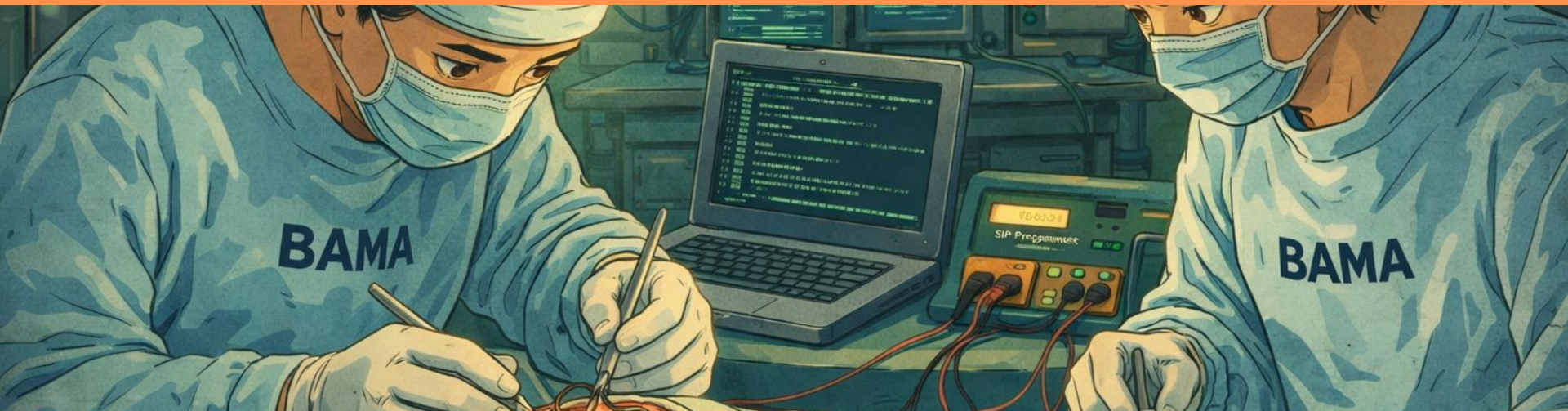
- Intel PIN
 - Dynamic binary instrumentation framework
 - Allows you to "intercept" x86 instructions at execution
 - Use it to figure out "expected" data:
 - Example: `if (input[36] == 0x3f) do_stuff();`

Your task

- Intel PIN
 - Dynamic binary instrumentation framework
 - Allows you to "intercept" x86 instructions at execution
 - Use it to figure out "expected" data:
 - Example: if (input[36] == 0x3f) do_stuff();
- You can run your PIN tool using something like this:

```
$PIN_ROOT/pin
-t obj-intel64/tainttool.so
-i input_file
--
./NLQBox -i <identity_file>
```


The analysis



Analysis

- Secret identity file is split into 4 parts
 - Each requires a different approach to recover
 - Roughly in increasing difficulty
 - Implementing more steps will reveal more of the secret

Analysis

- Secret identity file is split into 4 parts
 - Each requires a different approach to recover
 - Roughly in increasing difficulty
 - Implementing more steps will reveal more of the secret
 - **The 4 parts are not split in blocks** (not contiguous)

Key insights

1. Each input character (position in the file) is **only compared once**
2. You can "progressively" improve
 - Implementing more steps will reveal more of the secret
 - There are similar characters in different positions.
 - **Not necessarily in complete chunks**
 - **A new step maybe reveals characters completely scattered**
 - **Even if the previous step was a contiguous block**

Analysis steps

1. Comparisons through direct compare instructions
2. Comparisons through library functions
3. Comparisons with arithmetic evasion
4. Comparisons with advanced obfuscation

Step 1: direct comparisons

```
if ((msg[111]=='1') && (msg[112]=='.') && (msg[113]=='2')) {  
    /* version 1.2 -> perform further parsing */  
}
```

- The parser will continue only if msg contains "1.2"
- If msg is tainted, we will observe some cmp instructions between tainted values and untainted constants ('1', '.', '2' etc)
- Setting input at buffer position 111-113 to "1.2" triggers new code path

Step 1: direct comparisons

- Instrument direct cmp instructions
 - Can only track direct copies and comparisons
- It's not quite so simple, but more on that later...

Step 2: Library function comparisons

```
if (strncmp(&msg[32], "HTTP/", 5) == 0) {  
    /* handle HTTP request */  
}
```

- Comparisons can happen through well-known libraries
- An argument is tainted and compared against an untainted string
- Semantics & signature is known; use that knowledge

Step 3: Arithmetic evasion

```
eax = msg[5];    // eax contains tainted 'k' (153)
ebx = eax + 7;    // taint not propagated on arithmetic
                  // instruction
ebx = ebx - 7;    // ebx contains original 'k'
if (ebx == 'k') { ... }
```

- Need to propagate taint on arithmetic instructions
 - (XOR, ADD, SUB, etc...)

Step 4: Obfuscation

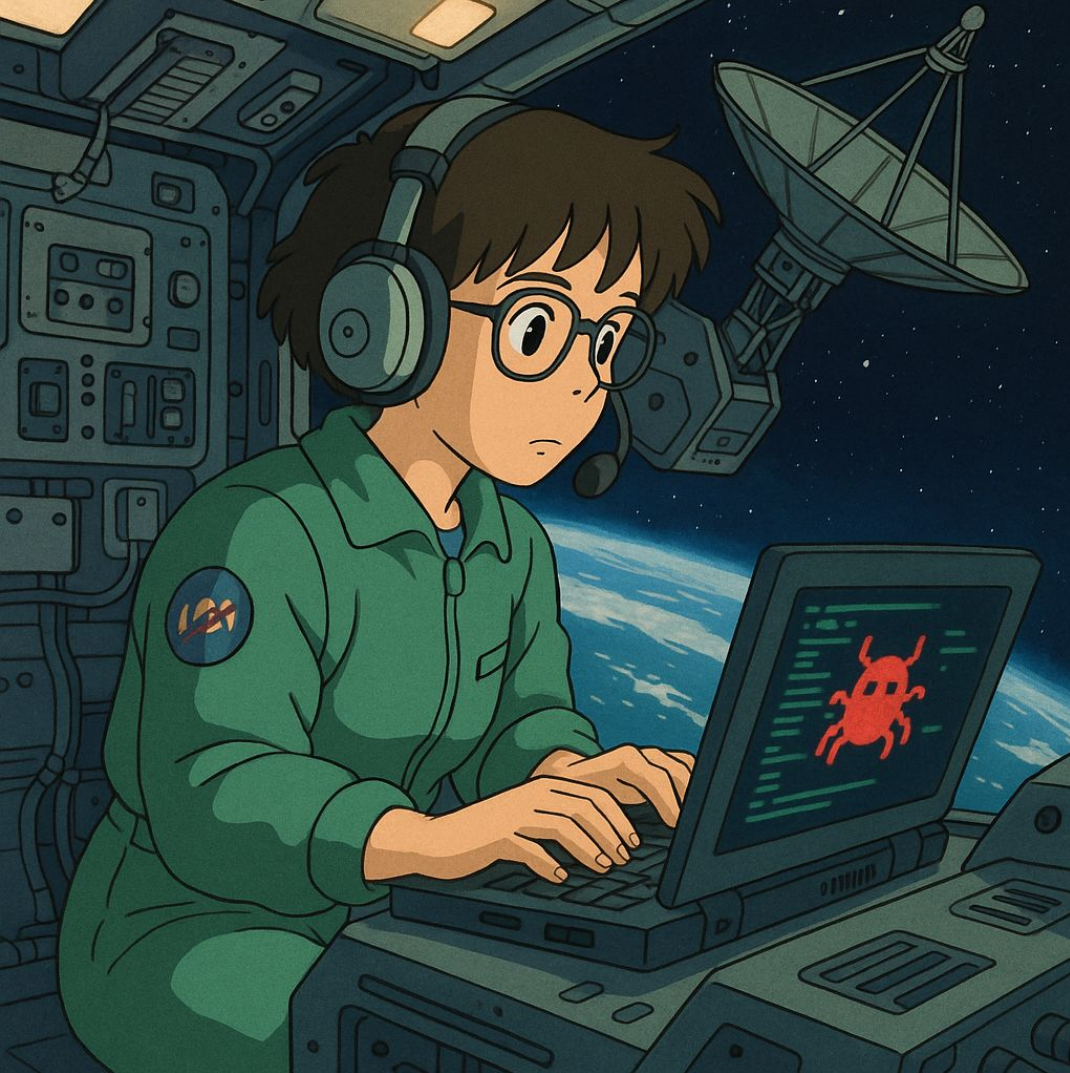
```
eax = msg[5];    // eax contains tainted 'k' (153)
ebx = eax + 7;    // ebx equals 'p' (153 + 7 = 160)
if (ebx == 'p') { ... }
```

- "Correct" character unknown at compare site
- Need to somehow "track" obfuscations that happened
 - Track tainted values; "reverse" obfuscations

Step 4: Obfuscation

```
eax = msg[5];    // eax contains tainted 'k' (153)
ebx = eax + 7;   // ebx equals 'p' (153 + 7 = 160)
if (ebx == 'p') { ... }
```

- "Correct" character unknown at compare site
- Need to somehow "track" obfuscations that happened
 - Track tainted values; "reverse" obfuscations
- **Do NOT try to generalize – focus on the secret!**



Caveats & Context

Step 1 caveat

```
mov ebx, input      // Tainted input into ebx
xor ebx, 0xc0ffee    // XOR with constant
cmp ebx, 0xc0ffe4    // Compare against expected
```

- Obfuscated equivalent of `input == 0xa`
- Can complicate step 1
 - ebx is tainted; xor does not yet propagate
 - cmp against untainted 0xc0ffe4
 - 0xc0ffe4 will be seen as part of secret

Step 1 caveat

```
mov ebx, input      // Tainted input into ebx
xor ebx, 0xc0ffee    // XOR with constant
cmp ebx, 0xc0ffe4    // Compare against expected
```

- For this step, "wash" taint on arithmetic operations
- Remove taint after (XOR, ADD, etc...) instructions
 - ebx will not be tainted at cmp
 - Value 0xc0ffe4 will be ignore

Context of Analysis

- CMP instructions are everywhere
- Code/external libs/etc may compare tainted values
 - e.g. if an internal print function compares a tainted value to -1, this doesn't necessarily mean that -1 is expected in the input
- I.e. the context of CMP instructions is important!

Focus on the goal

- Don't try instrumenting libc functions!
 - They use complex instructions and SSE registers
- Instead, propagate according to known semantics
 - E.g. you know what strcmp() does; manually set taint accordingly

Focus on the goal

- Don't try instrumenting libc functions!
 - They use complex instructions and SSE registers
- Instead, propagate according to known semantics
 - E.g. you know what strcmp() does; manually set taint accordingly
- Don't try implementing lots of x86 instructions!
 - **Limit to strictly needed**

Performance

- Make sure your analysis functions get inlined
- Avoid unnecessary work
 - Can you move logic from *analysis* to *instrumentation* routines?
- Do you need more than one pass?

Performance

- Make sure your analysis functions get inlined
- Avoid unnecessary work
 - Can you instrument routines?
- Do you need to instrument routines?

You can (and should) do it in one pass!



Be smart

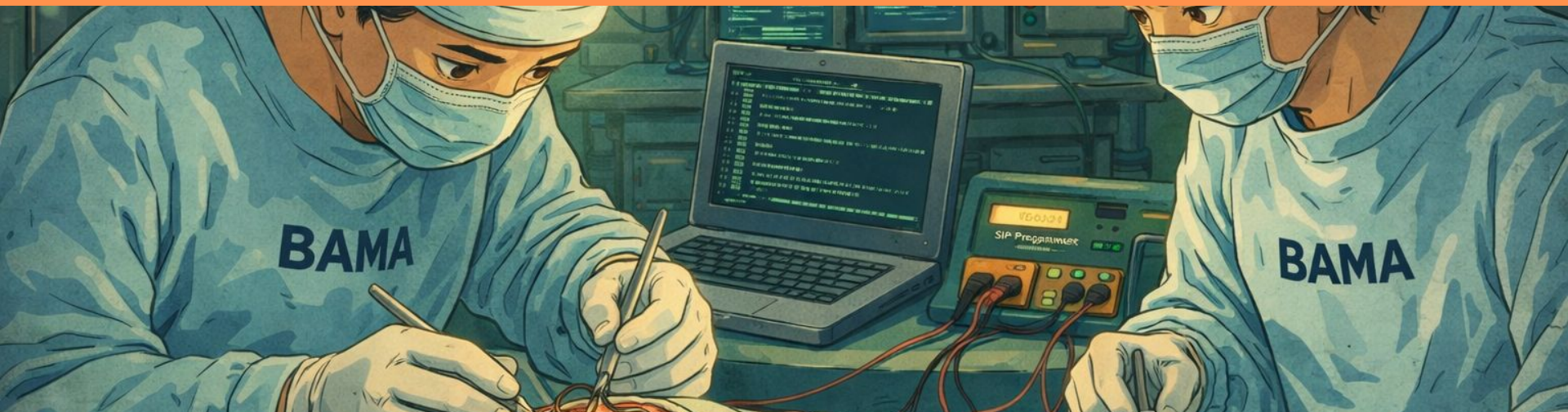
- Do not attempt static analysis
 - You do not need to work out what the binary is doing
- Write scripts/code for everything
 - Any language given we can run & understand it easily
- Focus on the task
 - No need to overgeneralise – you only need the secret!

Analysis steps

1. Comparisons through direct compare instructions
2. Comparisons through library functions
3. Comparisons with arithmetic evasion
4. Comparisons with advanced obfuscation

Advanced: bypassing anti-taint analysis

1 base point + 1 bonus point



Bypassing anti-taint analysis

- We heard the binary contains a second secret message
 - First secret message contains instruction how to trigger this path
- Entirely different internal processing!
 - Protection against dynamic taint analysis
- Seems to *wash taint*?
- Two classes of functions used for each byte:
 - Washers: real message?
 - Non-washers: fake message?

Approach

- Find big array with taint-washing functions in the binary
 - Manual inspection should work
- **Goal: Classify the functions in washers/non-washers; recover message!**
 - If you know the function SHOULD propagate taint, you can manually propagate it
 - If you know the function SHOULD NOT propagate taint, you can manually erase it
- Do whatever you want to recover the second secret message
 - Your (slightly adapted) PIN tool from the first part should do the job
 - Free to use something else :)

Classification

- There are different approach how to classify washer functions
 - You can choose a method you like :)
 - As long as it scriptable (can run automated)
- Recommend approach: dataflow-based classification
 - Detect for each function if the taint is propagated
 - Either via static analysis (e.g, Ghidra, Capstone, Zydis, Angr) or dynamic analysis (e.g., Intel PIN)

Warning

- This is an *highly advanced* part of the assignment
- No need to worry if you do not reach this part
 - A lot of fun if you try it :)
- In all-cases, first complete the first part before working on this

Grading

Exact criteria
&
submission requirements



Grading

- Step 1: compares through direct instructions → **2 pts**
- Step 2: compares through library functions → **2 pts**
- Step 3: compares with arithmetic evasion → **2 pts**
- Step 4: compares with advanced obfuscation → **1 pts**
- High performance tool → **1pt**
 - Do something smart to optimize your PIN tool (see lecture)
- Readability, automation, and convenience → **1 pt**
- *Advanced: classify washer functions* → **1 pt**
- *Advanced: retrieve second secret message* → **1 pt bonus**

Submission Guidelines

You need to deliver a **zip file** containing **at least**:

- TXT File **“input_secret.txt”** with the (parts of) the recovered identity file
 - If fully recovered, also: TXT file **“output_secret.txt”** with the secret message that is printed to screen when identify file is correct
- If you worked on the highly advanced part, also include
 - “washers.txt” and “fake.txt”, containing the classified function addresses
 - “input_secret2.txt” and output_secret2.txt”
- Script **“run.sh”** which runs your solution
- **README.md** explaining your submission & how to run it
 - Describe what, how, and why, and clearly explain how to reproduce
- Any **other** necessary files for running your submission

Your README.md

DO include...

- How to run your tool to get the secret input & output message
- For each of the four steps:
 - How did you support this step?
 - Why did you do it like this?
- Required steps to retrieve the full message
- Which performance optimizations did you do and why?

DON'T include...

- Inner workings of the binary
- An hourly log of everything you did
- Everything that didn't work
- How you received your binary
- Why you love PIN
- Why you hate the Pin User guide

Submission Guidelines

A friendly reminder:

- **We assume pin 3.31 (EOL) (\$PIN_ROOT)**
 - Framework does not support 4.* yet!
- Make sure you include all necessary files
- Use file extensions (.sh/.py/etc.)
- Use (environment) variables instead of hard coded variables

Submission Guidelines

Use (environment) variables

- Do not hardcode tool locations (use `$PIN_ROOT`, `$GHIDRA_HOME`)
 - And thus do not set them in your script!
- **For example, you can do instead:**
 - `PIN_ROOT="${PIN_ROOT:-/opt/pin}"`
 - (it will only set `PIN_ROOT` if it is not set)

Submission Guidelines

- Submission **must** be through Canvas & CodeGrade
- Deadline: Tuesday the 12th of May 2026 (15:30 CEST)
- Delay penalty: 1 point per 24 hours (strict)

Problems?

- Is something wrong with your binary?
- Are you having any issues?

Reach out to us in time!

(and make use of the office hours)

Extra

Download Pin 3.31 here:

- <https://software.intel.com/sites/landingpage/pintool/downloads/pin-external-3.31-98869-gfa6f126a8-gcc-linux.tar.gz>

Pin 3.31 User guide:

- <https://software.intel.com/sites/landingpage/pintool/docs/98869/Pin/doc/html/index.html>

Running an ARM CPU?

The binary is a x86_64 binary. If you are using a non-x86_64 CPU (e.g., ARM), you can request an X86 VM.

REQUESTING THE VM:

To request a Linux VM, please send a email as follows:

- Start your email title with "[BAMA Linux VM]"
- Include your vundet-id
- Include your ssh-key
- Send to a.markovic@vu.nl with bama@vusec.net in CC

NEURALINQ

to your dump

Good luck!

ADVANCED BRAIN EXPLOIT VECTOR
BRAIN-COMPUTER
INTERFACE

CMD: HACK_BRAIN_FUNC

SYSTEM COMPROMISED

HACKER_ID: [REDACTED]

ACCESS: ALL

BRAIN_FUNCTION_OVERRIDE: ACTIVE

ROOT ACCESS GRANTED

COMMAND_OVERRIDE_DET

NEURAL SUBJUGATION

NEUROL_OPERATION

NEURAL_COVERRITEG_DET

END_OF_FREE_WILL

BRAIN_EXPLOIT_VECTED

0101010001000101

1001101011001

0110001001010100

01001010011101

0100000010101

010101010001

01000101010101

CONTROL_UPLOAD: 91%